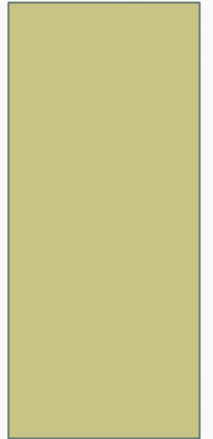


EC-200 DATA STRUCTURES

LECTURE 11



TREE DATA STRUCTURE

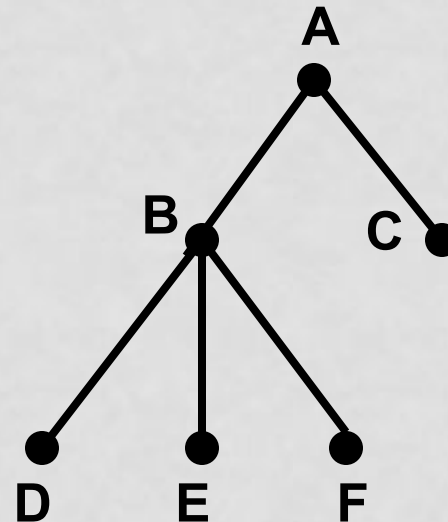
- **Introduction**

- The Data Organizations Presented Earlier are Linear in That Items are One After Another
- ADTs in This Lecture Organize Data in a **Non-Linear, Hierarchical Form**, Whereby an Item can Have More Than one Immediate Successor

DEFINITION

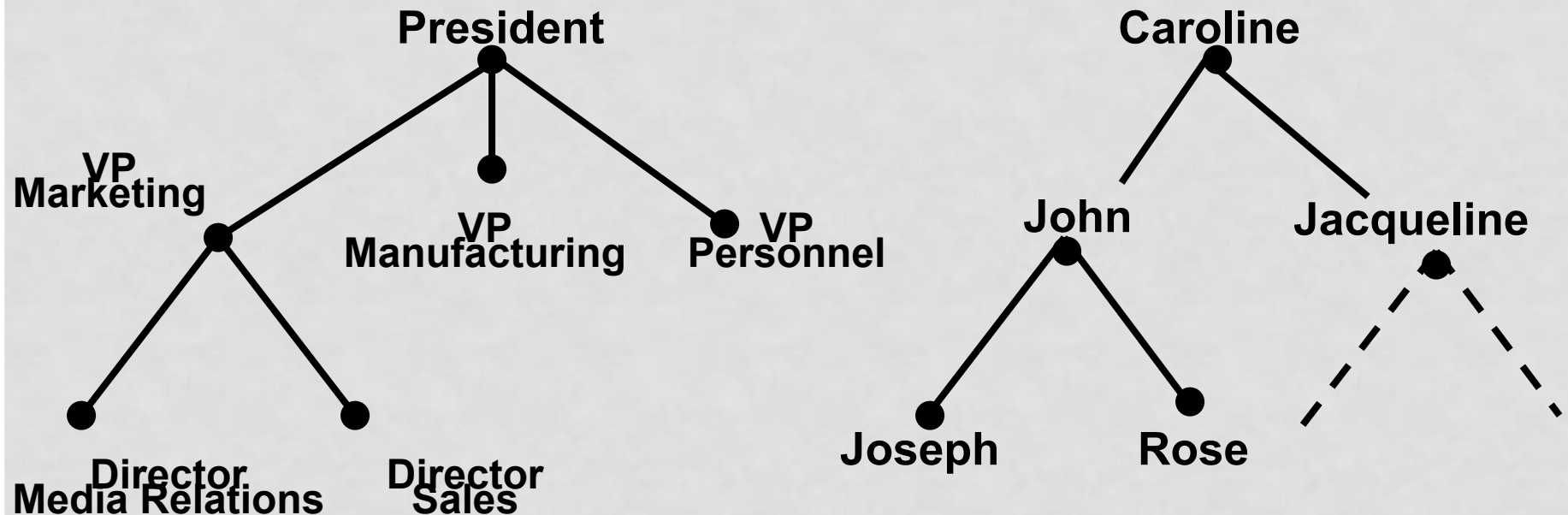
- A (general) tree is a collection of Nodes and edges such that
 1. There is a **single node** from which paths to other nodes emerge; this node is called the **root**
 2. There is a single, **unique path** to reach any node of the tree

A General Tree



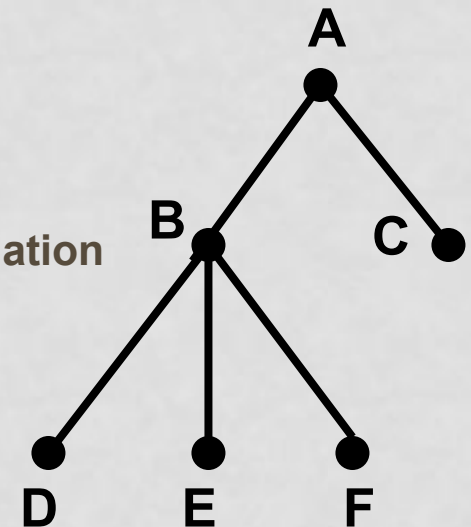
APPLICATIONS

Because Trees are Hierarchical in Nature, You Can Use Them to Represent Information That Itself is Hierarchical in Nature, For Example **Organization Charts** and **Family Trees**



TERMINOLOGY

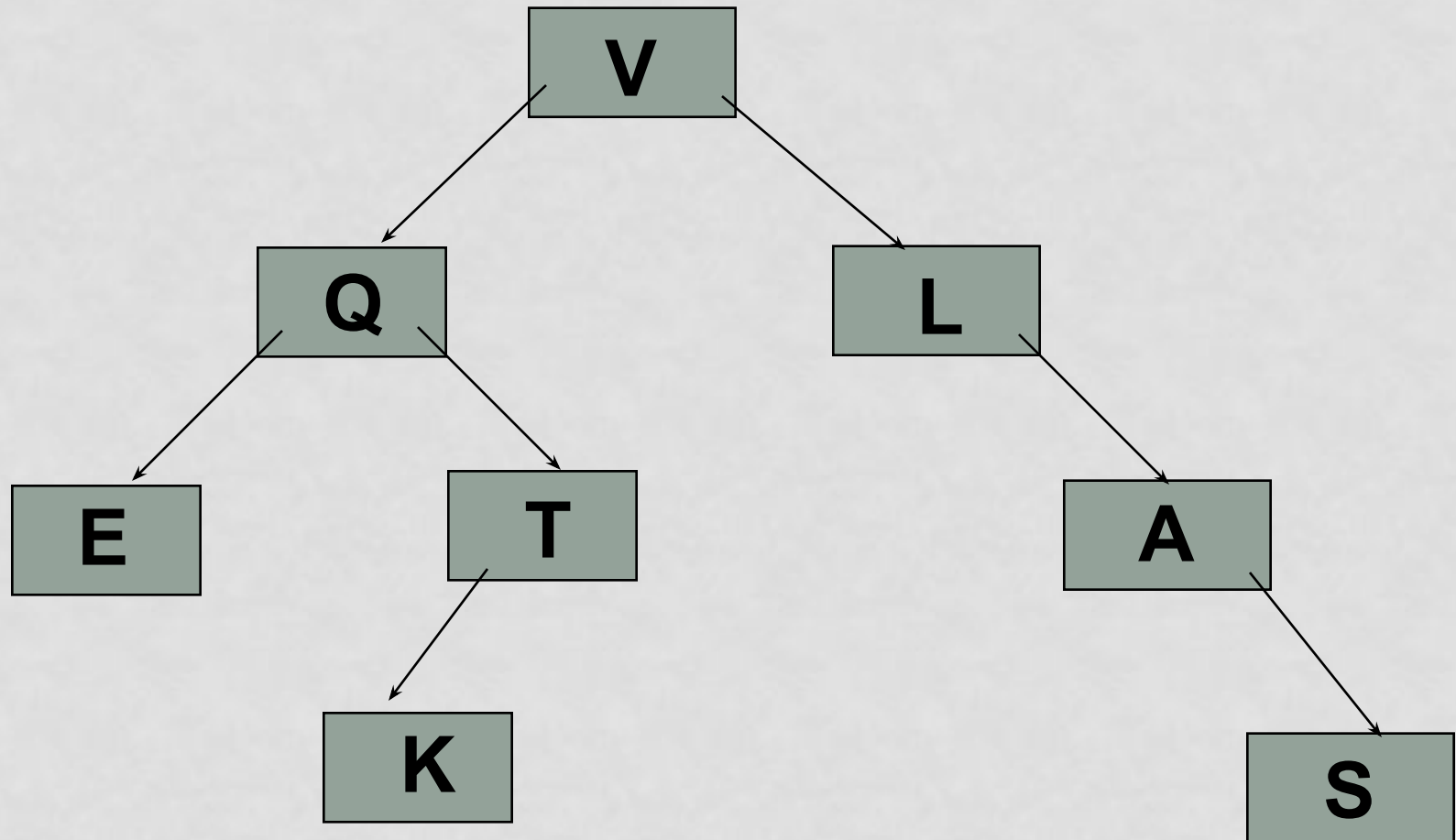
- If there is an Edge from Node n to Node m, Then n is the **Parent** of m, and m is a **Child** of n
- Each Node in a Tree Has at Most **One Parent**, and the Root of the Tree has **no Parent**
- Children of the Same parent are Called **Siblings**
- A Node That Has no Children is Called a **Leaf**. Non-leaf nodes are called **internal nodes**
- **Example Figure:**
 - Node A is the **Root** of the Tree,
 - Nodes B and C are **Children** of Node A
 - A is an **Ancestor** of D, and Thus, D is a **Descendant** of A
 - B and C are Not Related by Ancestor and Descendant Relation



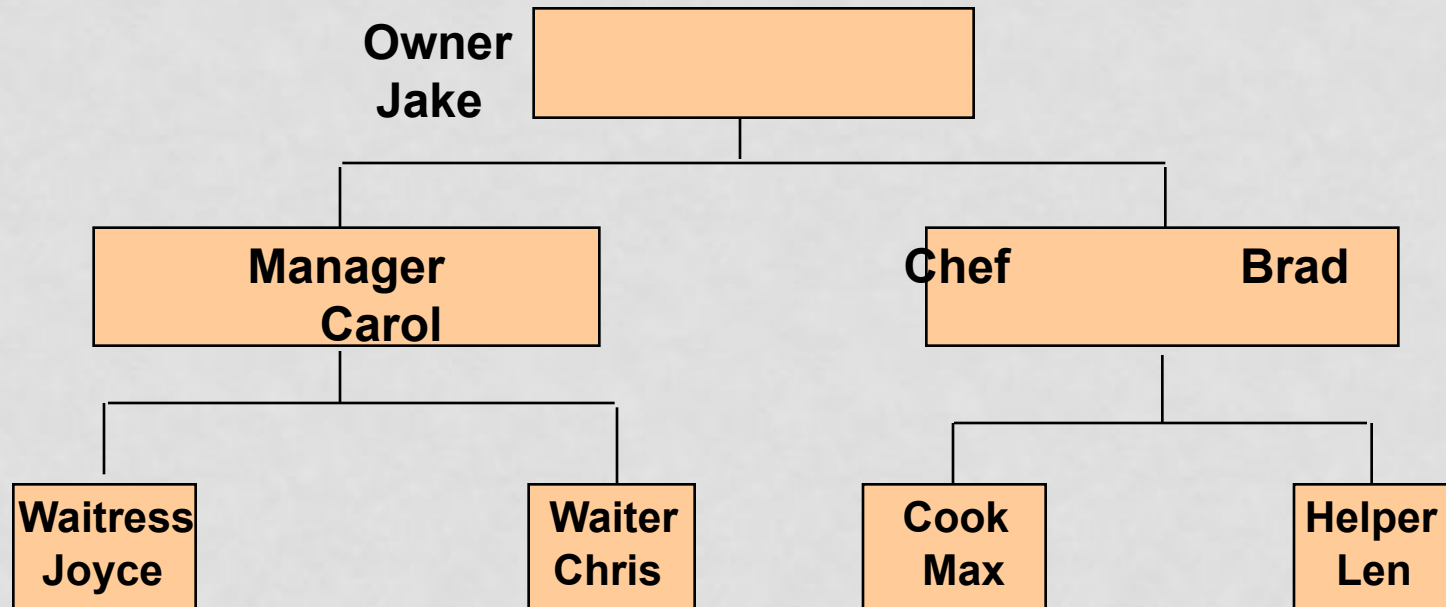
TERMINOLOGY

- **Binary Tree**: a finite set of elements that is either empty or is partitioned into three disjoint subsets.
 - The first subset contains a single element called the **root** of the tree
 - The other two subsets are themselves binary trees, called the **left** and **right sub-trees** of the original tree
 - Each element of a binary tree is called a **node**

A BINARY TREE

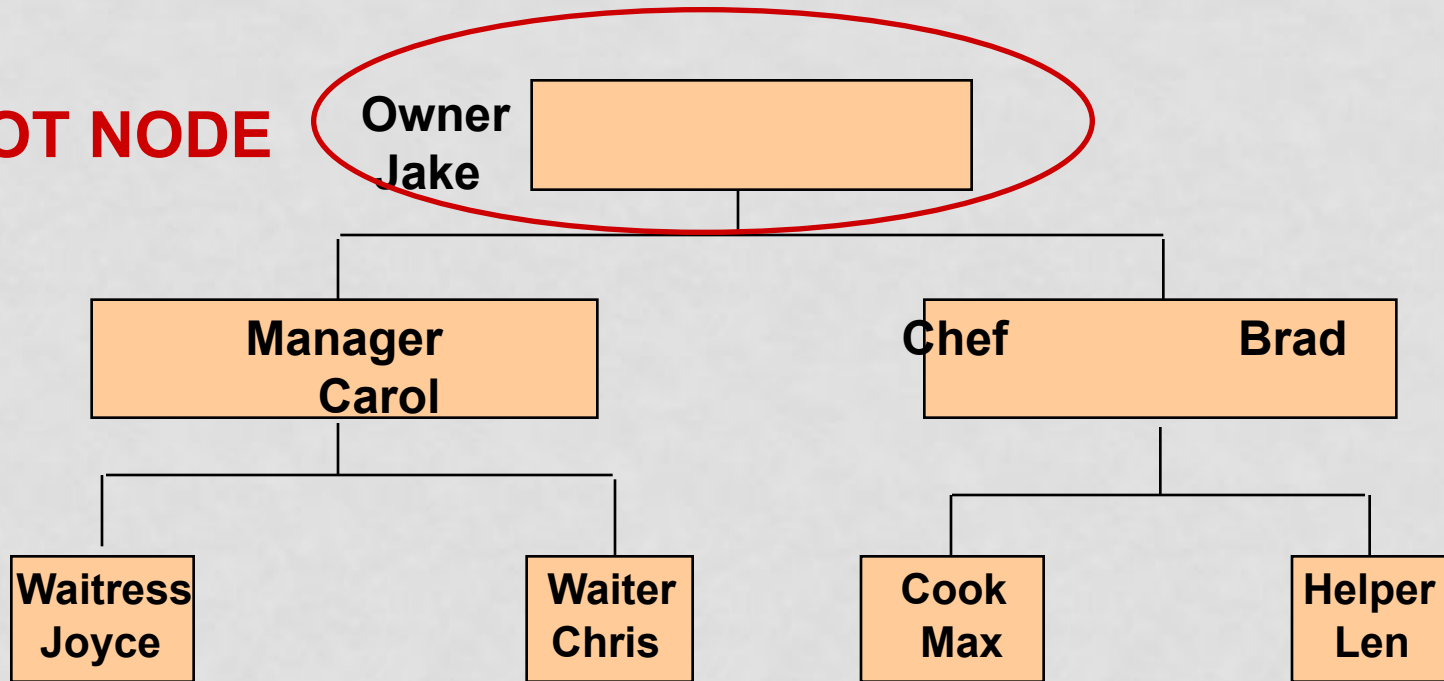


Example: Jake's Pizza Shop

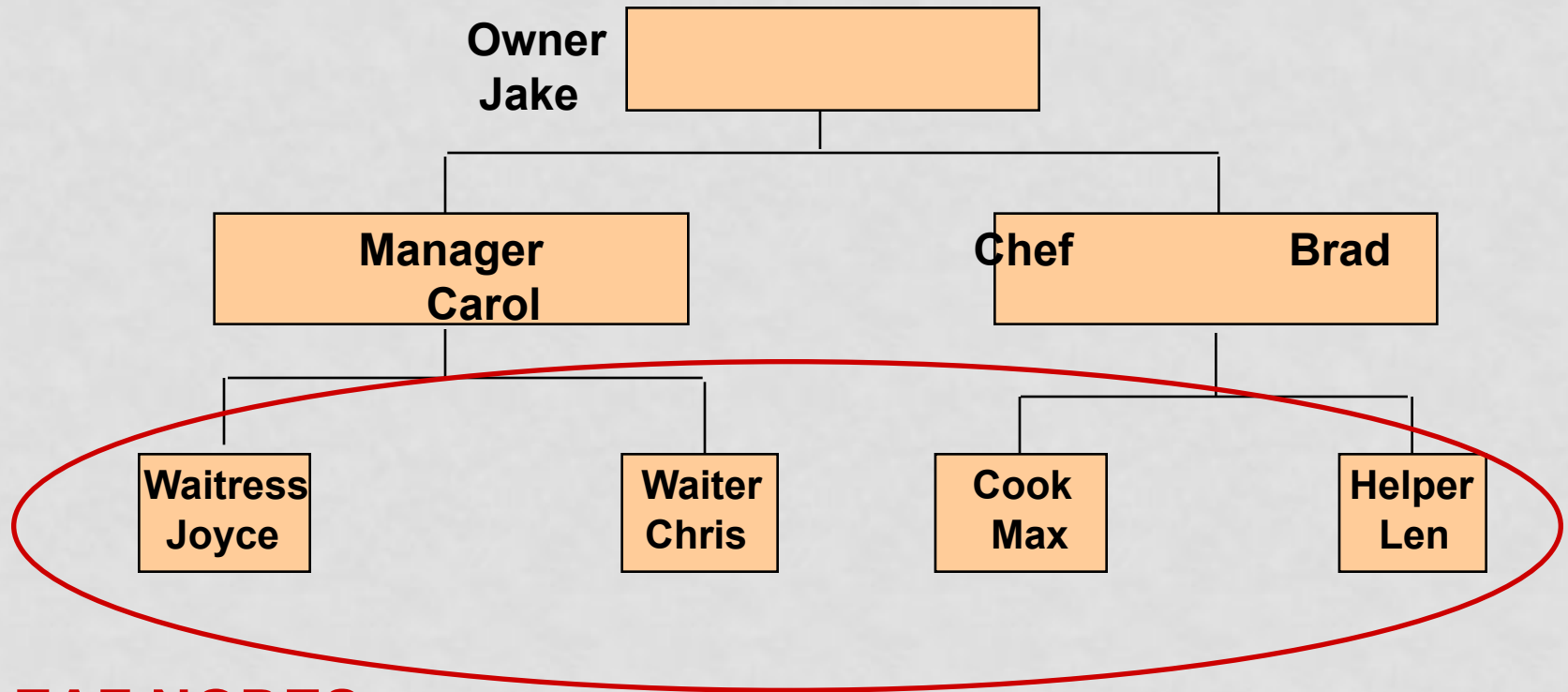


A Tree Has a Root Node

ROOT NODE



Leaf nodes have no children

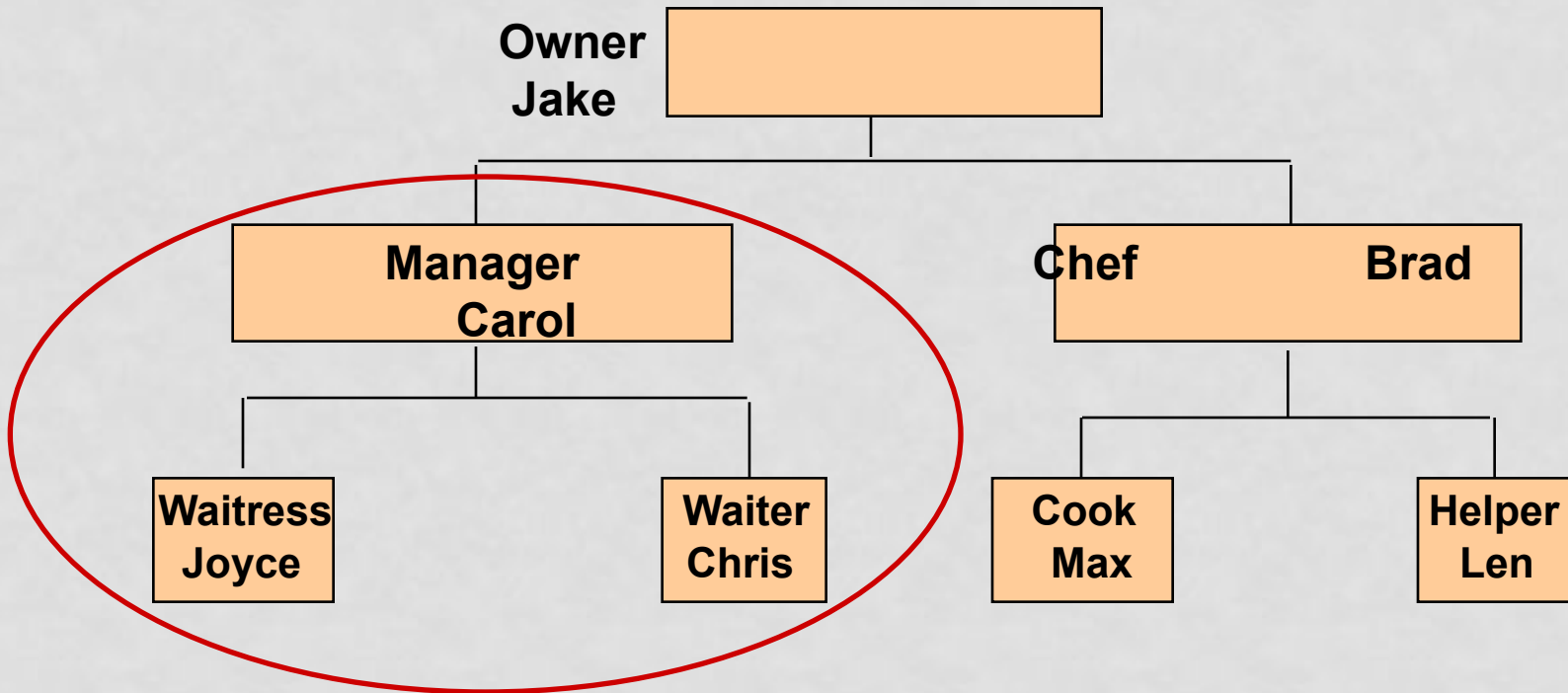


LEAF NODES

SUBTREES

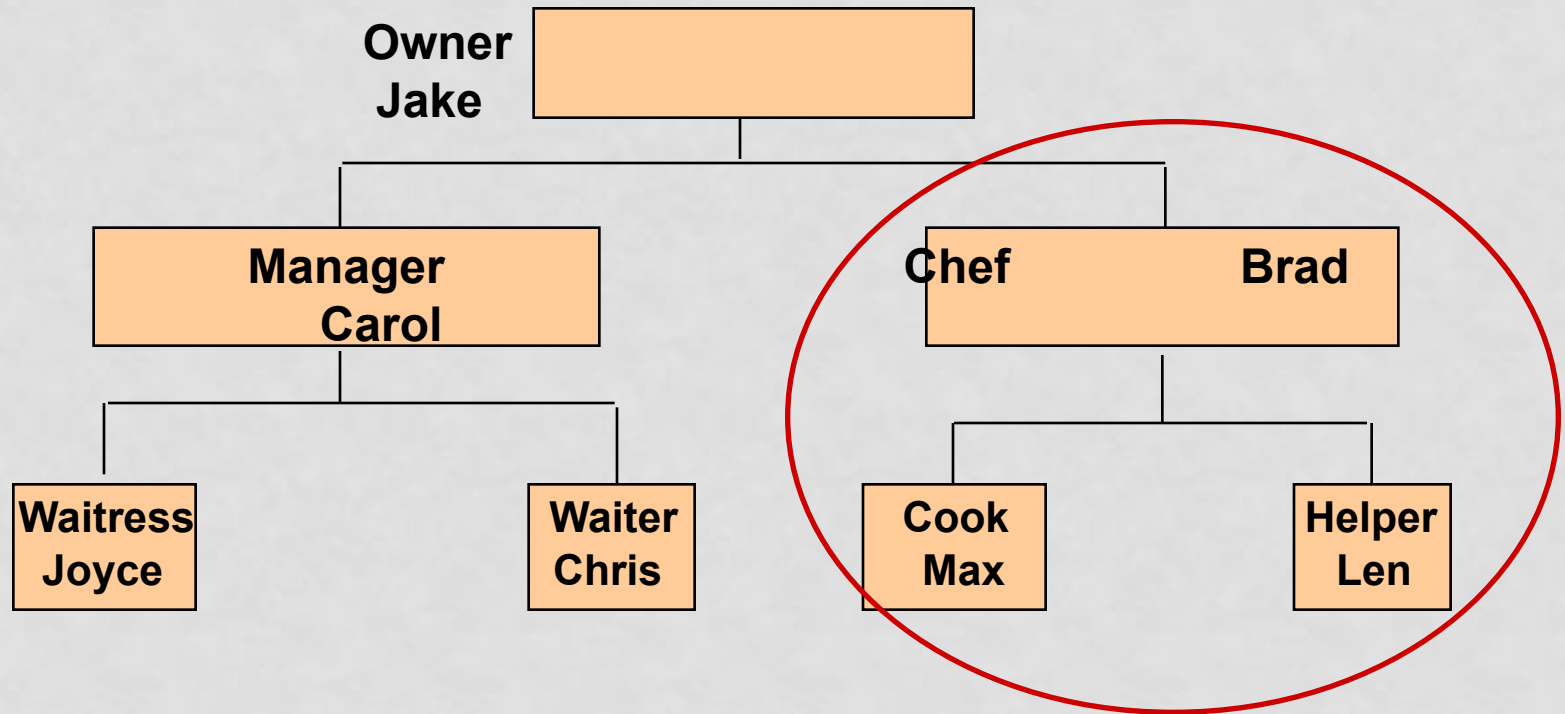
- A **subtree** rooted at u is the tree formed from u and all its descendants.

A Subtree



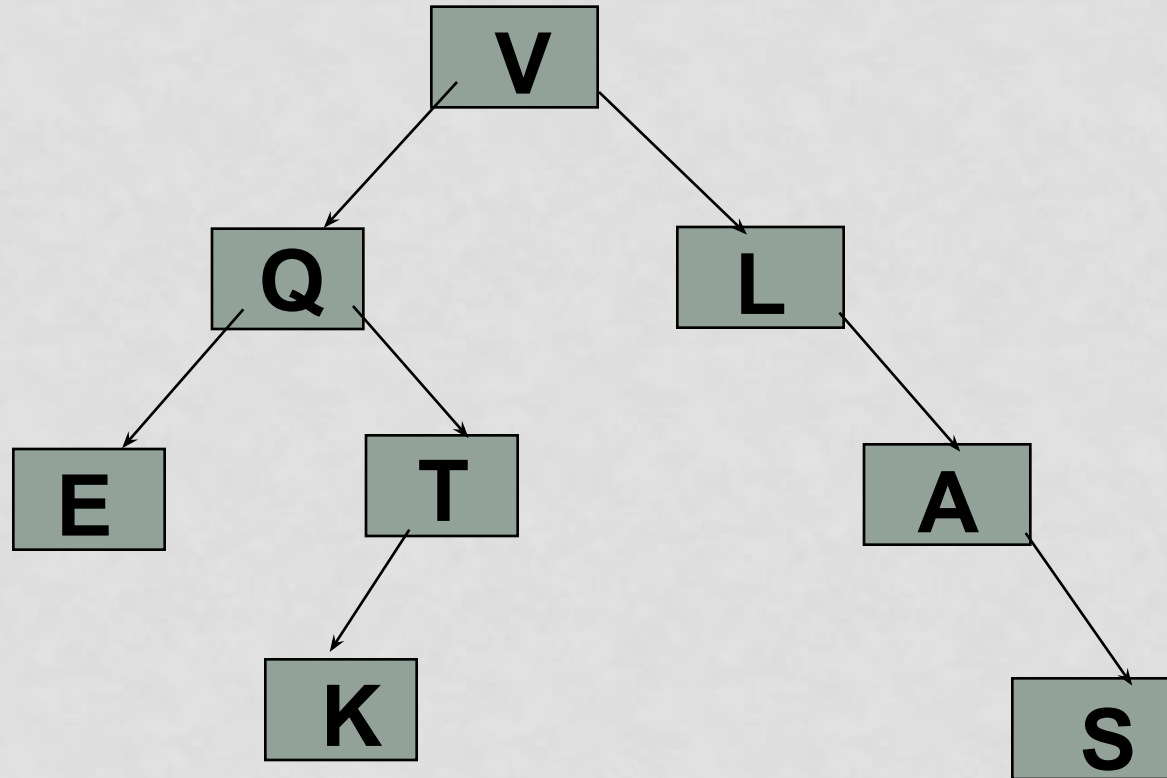
LEFT SUBTREE OF ROOT NODE

Terminology

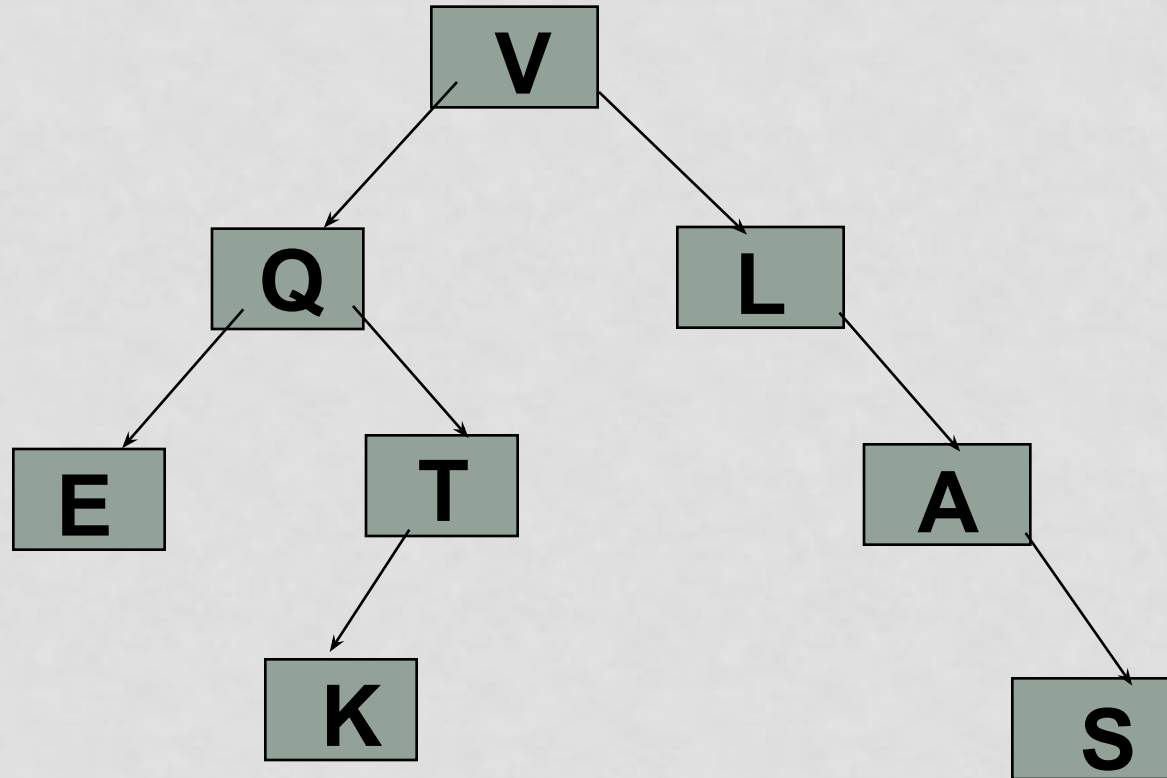


**RIGHT SUBTREE
OF ROOT NODE**

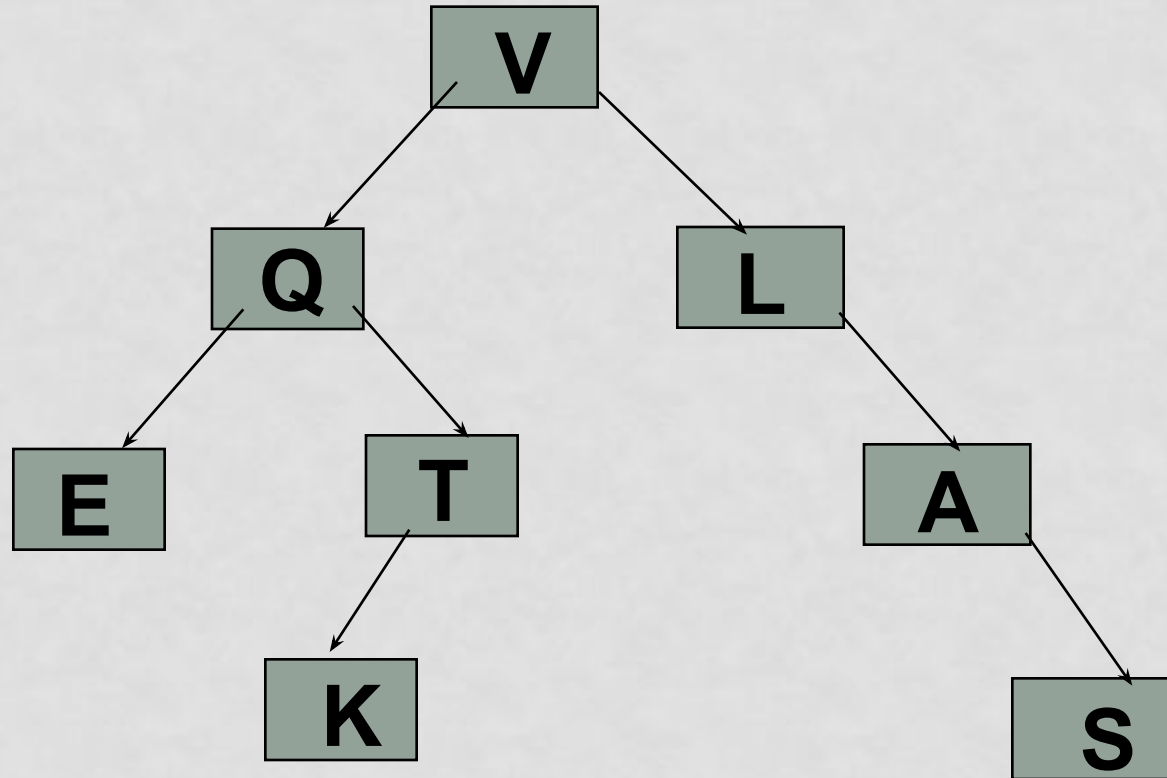
EXAMPLE: HOW MANY LEAF NODES?



EXAMPLE: HOW MANY DESCENDANTS OF Q?

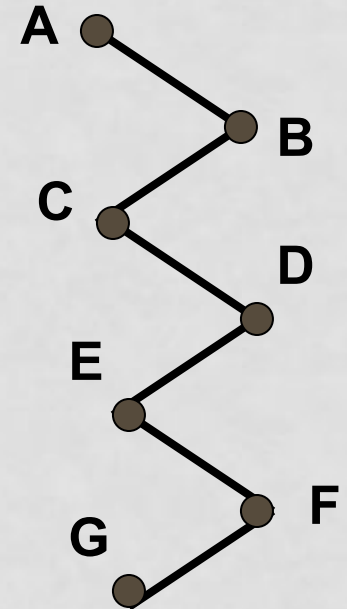
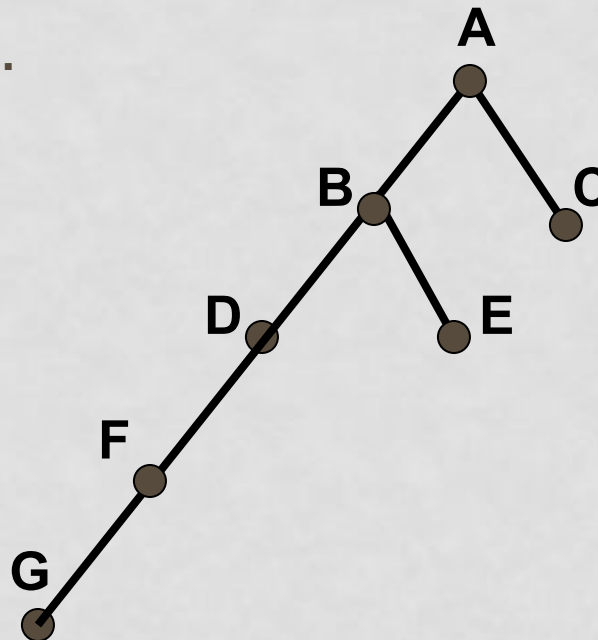
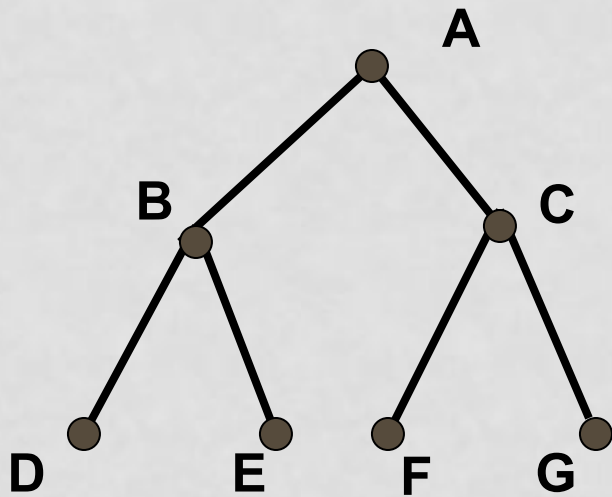


EXAMPLE: HOW MANY ANCESTORS OF K?



TERMINOLOGY

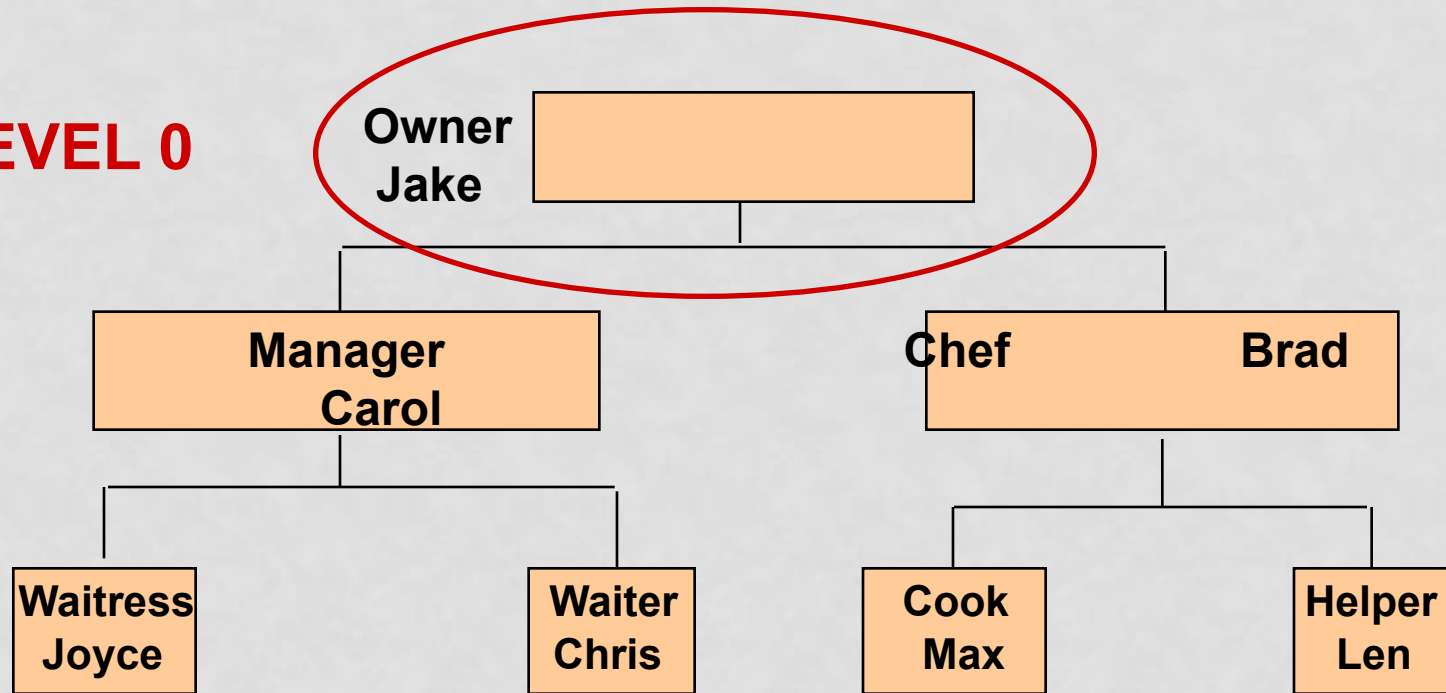
- Trees come in Many Shapes.



- **Level of a Node n**
 - If n is the **Root**, it is at **level 0**.
 - If n is not the Root, its **Level is 1 Greater Than the Level of its Parent**
- **Depth (also called Height) of Tree**
 - Length of the Longest Path (in terms of number of edges) From the Root to a Leaf
 - Maximum level of any leaf in the tree

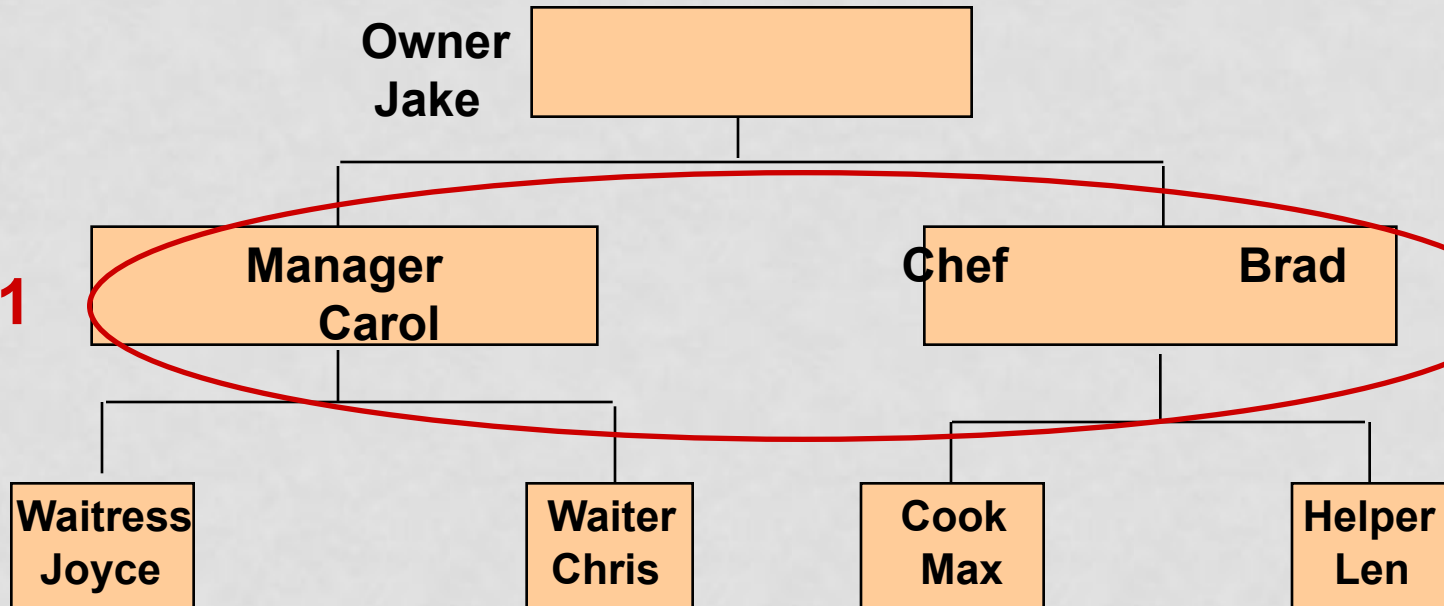
A Tree Has Levels

LEVEL 0

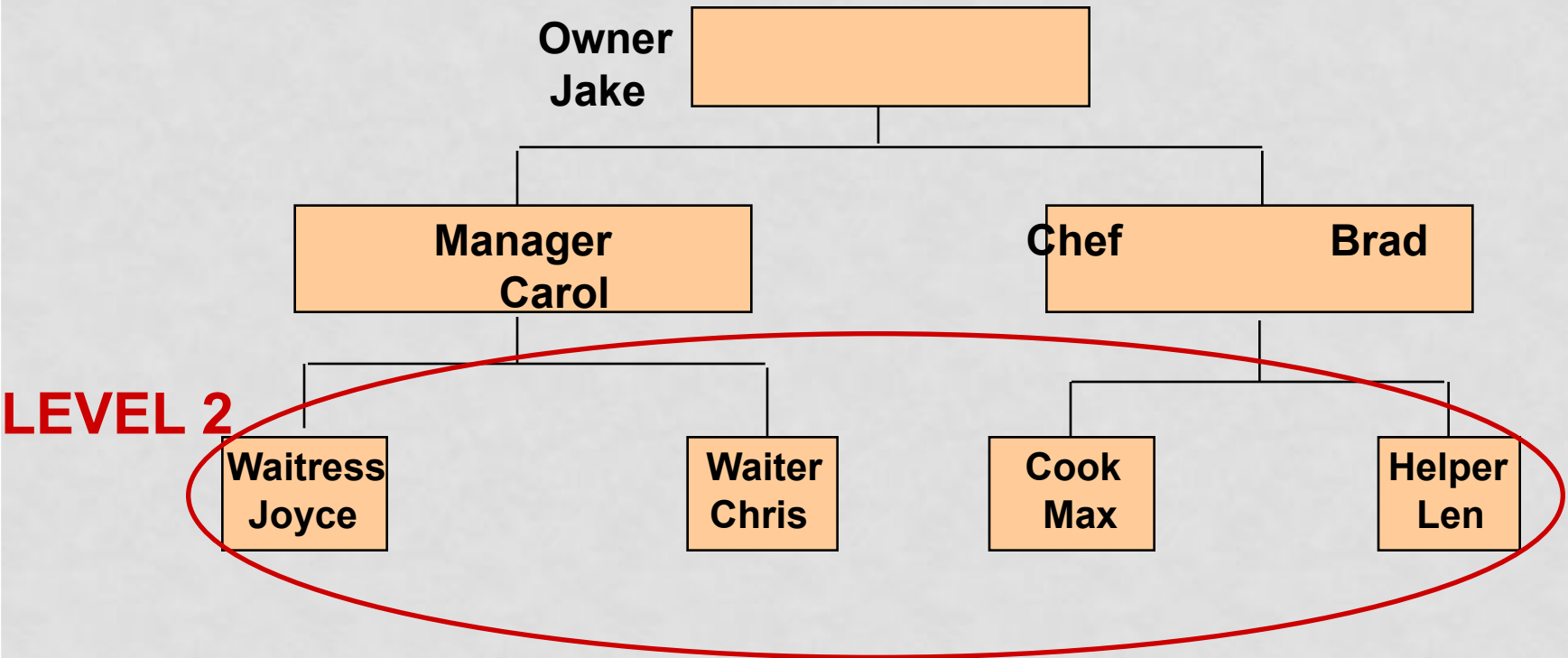


Level One

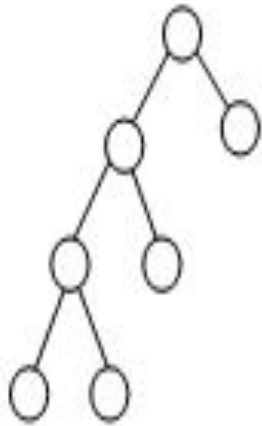
LEVEL 1



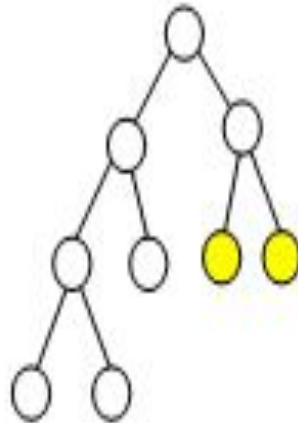
Level Two



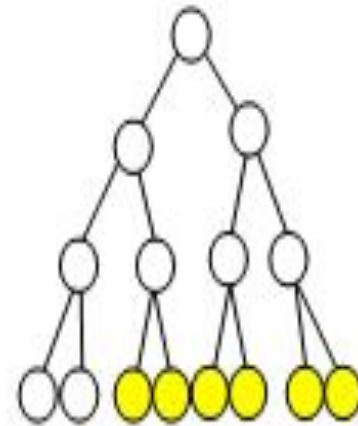
FULL, COMPLETE, AND PERFECT BINARY TREE



full



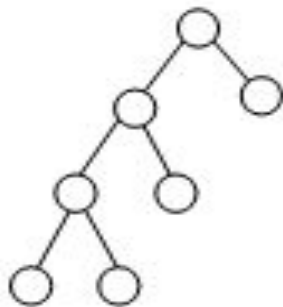
complete



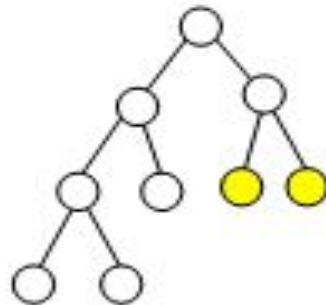
perfect

FULL, COMPLETE, AND PERFECT BINARY TREE

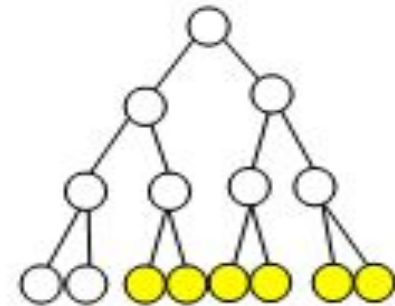
- If every node has either 0 or 2 children, a binary tree is called full.
- If the lowest $d-1$ levels of a binary tree of height d are filled and level d is partially filled from left to right, the tree is called complete.
- If all d levels of a height- d binary tree are filled, the tree is called perfect.



full

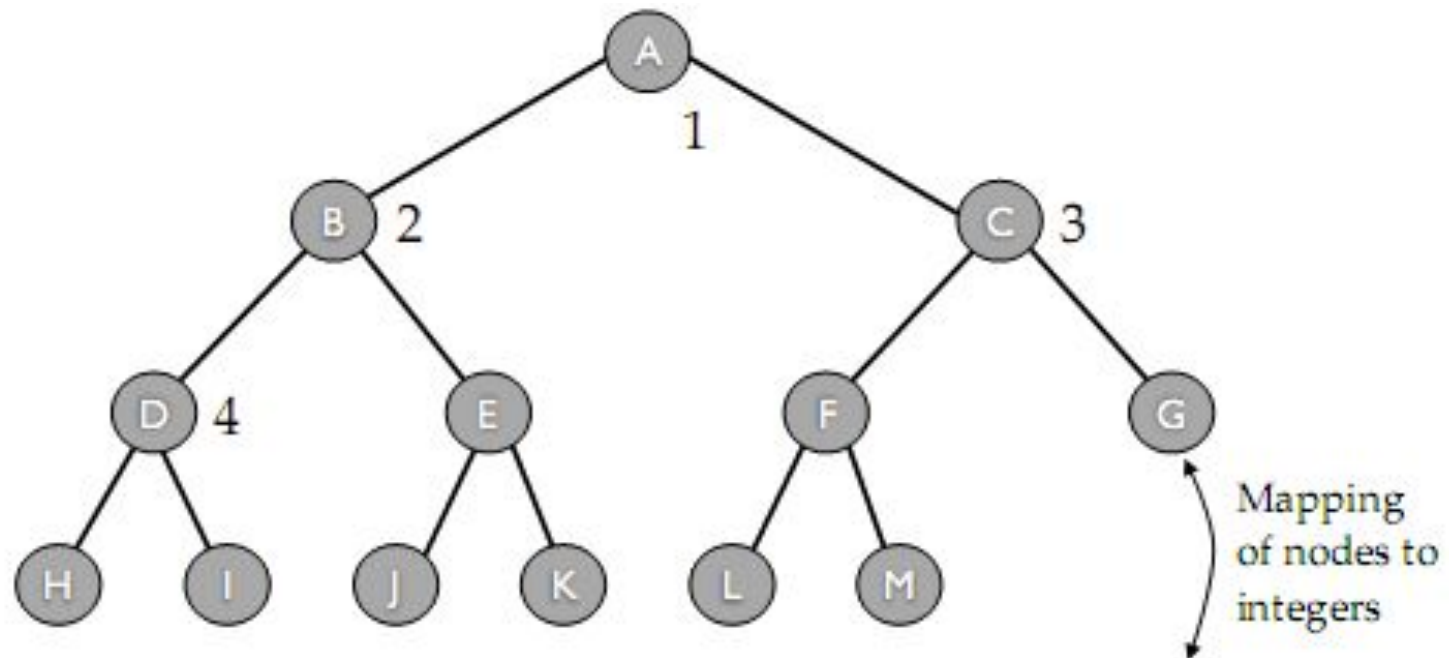


complete



perfect

ARRAY IMPLEMENTATION FOR COMPLETE BINARY TREES



A	B	C	D	E	F	G	H	I	J	K	L	M		
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

$\text{left}(i): 2i$ if $2i \leq n$ otherwise 0

$\text{right}(i): (2i + 1)$ if $2i + 1 \leq n$ otherwise 0

$\text{parent}(i): \lfloor i/2 \rfloor$ if $i \geq 2$ otherwise 0

TRAVERSALS OF A BINARY TREE

- When Traversing any Binary Tree, the Algorithm has Three Choices of When to Visit a Node r :
 - It Can Visit r Before it Traverses both of r 's Subtrees
 - It Can Visit r After it Has Traversed r 's Left Subtree T_L But Before it Traverses r 's Right Subtree T_R
 - It Can Visit r After it Has Traversed Both of r 's Subtrees
- These Traversals are Called **Preorder, Inorder, and Postorder** traversals Respectively

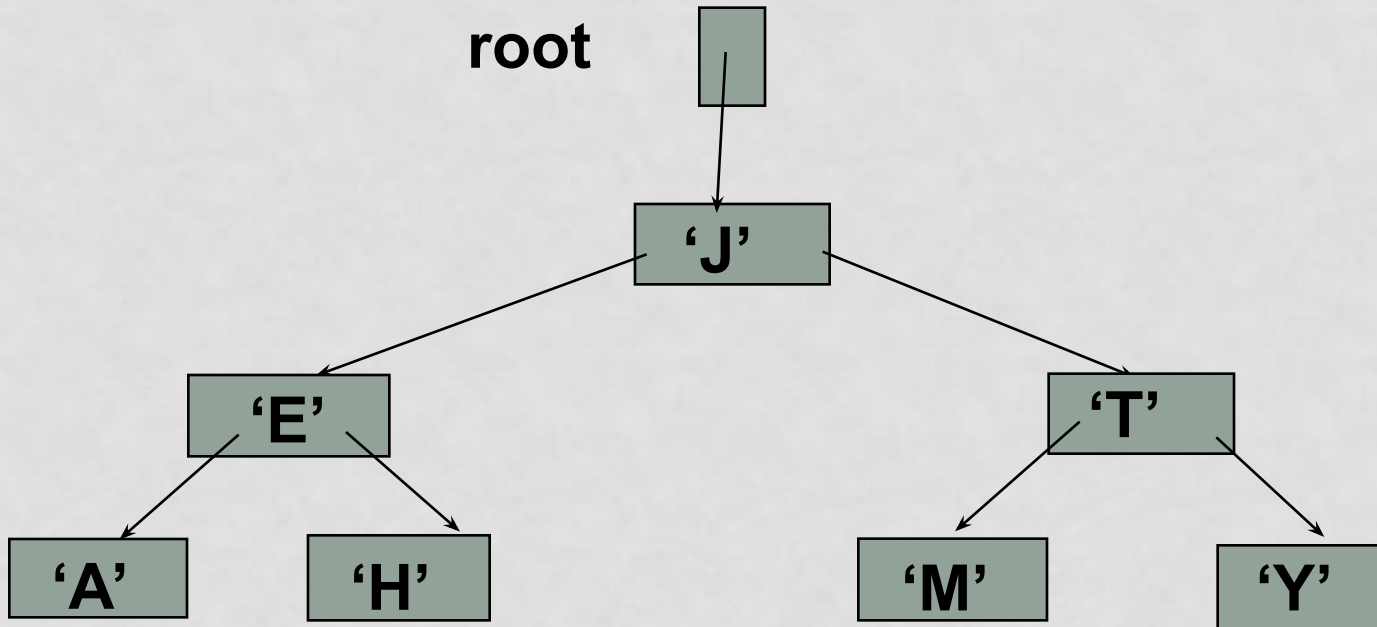
TRAVERSALS OF A BINARY TREE

Preorder Traversal

- 1. Visit the root**
- 2. Traverse the Left subtree in Preorder**
- 3. Traverse the Right subtree in Preorder**

Notice recursive nature of definition

PREORDER TRAVERSAL: **J E A H T M Y**



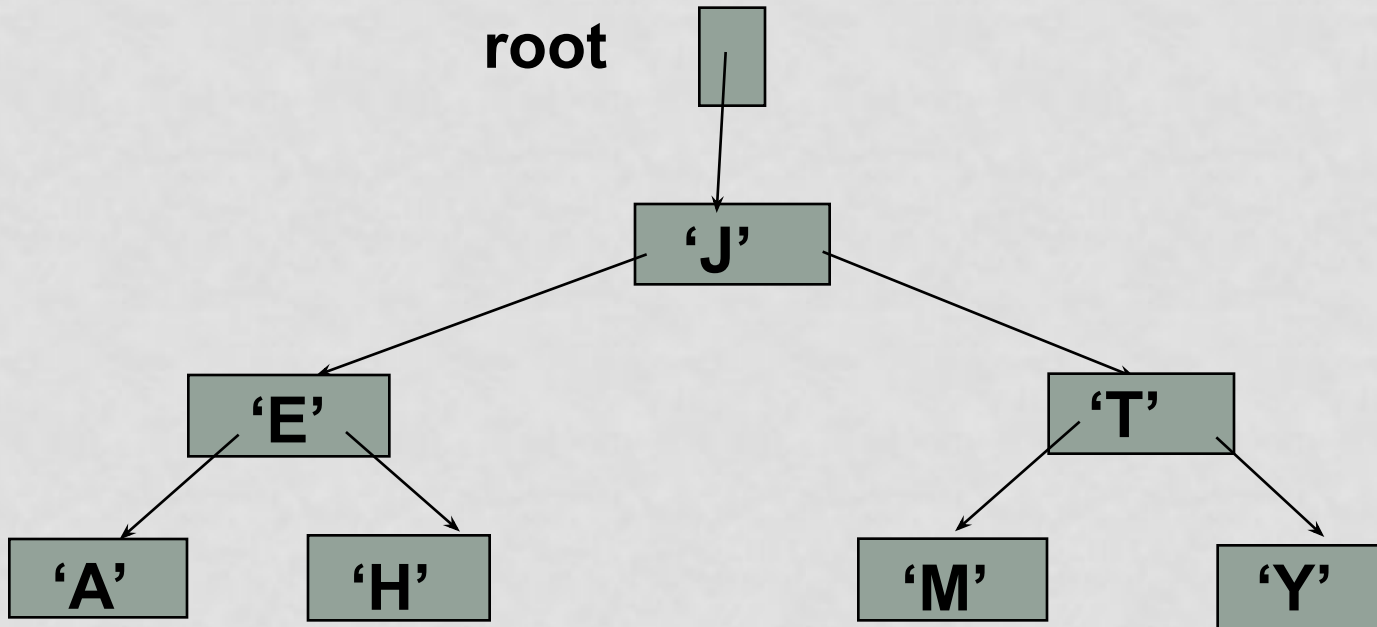
TRAVERSALS OF A BINARY TREE

Inorder Traversal

- 1. Traverse the Left subtree in Inorder**
- 2. Visit the root**
- 3. Traverse the Right subtree in Inorder**

Notice recursive nature of definition

INORDER TRAVERSAL: **A E H J M T Y**



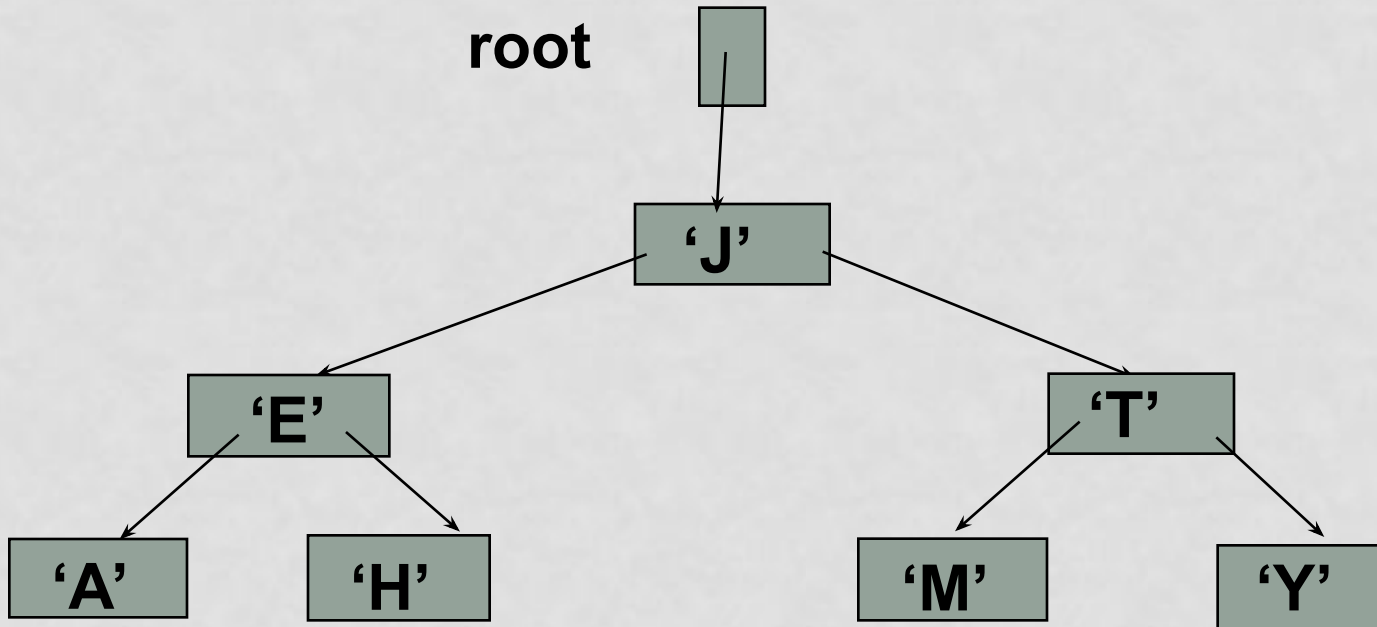
TRAVERSALS OF A BINARY TREE

Post order Traversal

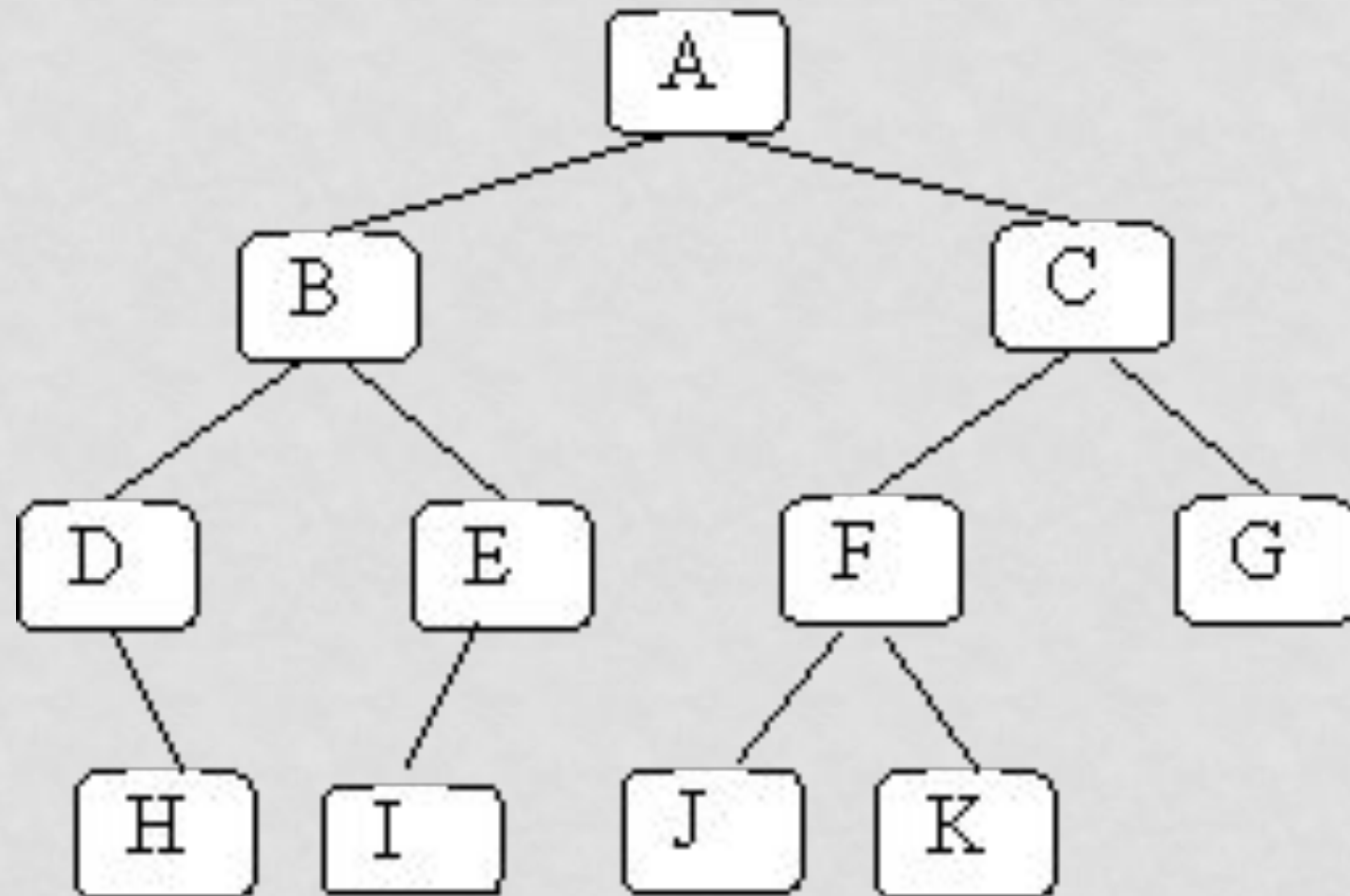
- 1. Traverse the Left subtree in Postorder**
- 2. Traverse the Right subtree in Postorder**
- 3. Visit the root**

Notice recursive nature of definition

Postorder Traversal: A H E M Y T J



BINARY TREE



TRAVERSALS

- Preorder traversal -- A B D H E I C F J K G
- Postorder traversal -- H D I E B J K F G C A
- In-order traversal -- D H B I E A J F K C G

READING

- Chapter 10, Data Abstraction and Problem Solving using C++, Walls & Mirrors 3rd ed.